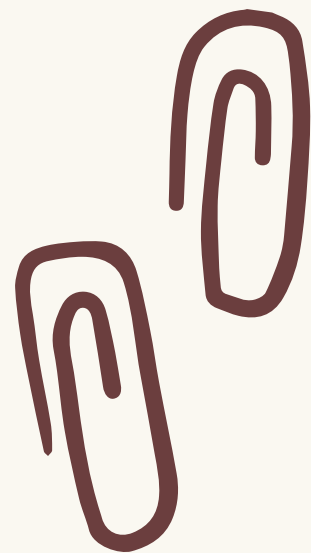




FinOptix

Summer Project '25

MENTORS

Keshav Bansal • Kushagra Tiwari
Aarush Singh • Ankit Kumar



CAPM



Overview

The Capital Asset Pricing Model (CAPM) is a financial model that describes the relationship between the expected return of an investment and its risk. It is widely used in finance to assess the expected return on assets, particularly stocks, given the risk relative to the market.

Formula Overview :

$$E(R_i) = R_f + \beta_i \cdot (E(R_m) - R_f)$$

Expected Return ($E(R_i)$): This is the return an investor expects from an asset.

Risk-Free Rate (R_f): This represents the return on a risk-free investment, typically government bonds. It's the baseline return that investors can earn with no risk.

Beta (β_i): Beta measures the sensitivity of an asset's return relative to the return of the overall market.





LIMITATIONS OF CAPM

- **CAPM explains returns using only one factor (market risk), ignoring other proven drivers like size, value, and momentum.**
- **Real-world data often contradicts CAPM predictions — high-beta stocks don't always yield higher returns, and low-beta stocks sometimes outperform.**

Fama French 3 Factor Model

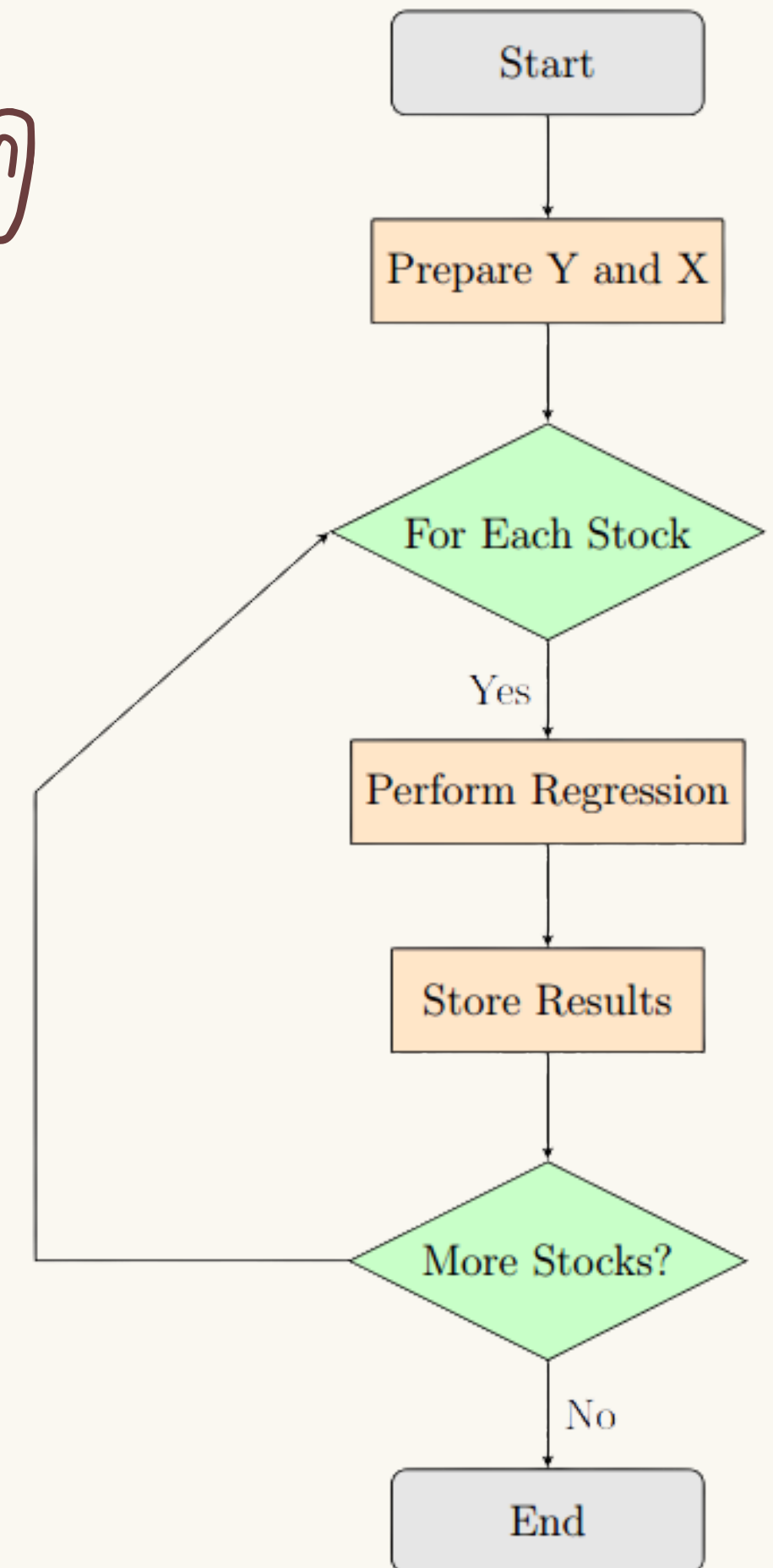
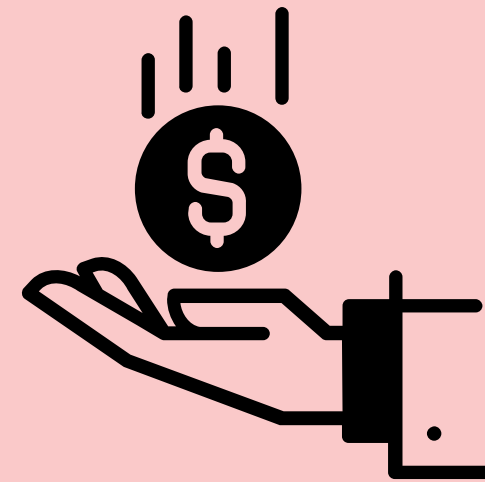
Overview

The Fama-French Three-Factor Model extends the traditional Capital Asset Pricing Model (CAPM) by adding two additional factors—size (SMB) and value (HML)—to explain stock returns more effectively than using market risk alone.

Formula Overview

$$R_p - R_f = \alpha + \beta_m (R_m - R_f) + \beta_{smb} \cdot (SMB) + \beta_{hml} \cdot (HML)$$

- Where:
- R_p : Return of the asset
- R_f : Risk-free rate
- R_m : Market return
- SMB: "Small Minus Big" — captures size
- HML: "High Minus Low" — captures value
- α : Intercept (abnormal return not explained by factors)





Breaking of the 3 factors

1. Market Risk Premium($R_m - R_f$)

- Captures systematic market risk — similar to CAPM.
- Indicates how sensitive an asset is to general market movements.
- A higher beta here means stronger correlation with market fluctuations.

2. SMB (Small Minus Big)

- Captures the size effect — smaller companies tend to outperform larger ones in the long run.
- A positive coefficient (β_{smb}) means the stock behaves more like a small-cap.
Constructed as:
- $SMB = \text{Average return of small stocks} - \text{Average return of big stocks}$

3. Hml (high Minus low)

- Captures the value effect — value stocks (high book-to-market ratio) tend to outperform growth stocks.
- A positive coefficient (β_{hml}) means the stock is more "value-oriented."
Constructed as:
- $HML = \text{Average return of high book-to-market stocks} - \text{Low book-to-market stocks}$

```

tickers = [
    "AAPL", "MSFT", "AMZN", "NVDA", "GOOGL", "TSLA", "BRK-B", "JNJ", "WMT", "JPM",
    "V", "PG", "UNH", "HD", "BAC", "MA", "DIS", "XOM", "KO", "PFE",
    "CVX", "MRK", "PEP", "ABBV", "TMO", "LLY", "AVGO", "CSCO", "COST", "DHR",
    "ACN", "MCD", "NKE", "WFC", "NEE", "INTC", "TXN", "LIN", "MDT", "HON",
    "LOW", "PM", "IBM", "UPS", "QCOM", "AMGN", "ORCL", "RTX", "NFLX", "BA"
]
portfolio_monthly=yf.download(tickers,period="max",interval="1mo")["Close"].pct_change()*100
portfolio_monthly.dropna(inplace=True)
portfolio_monthly.index = portfolio_monthly.index.strftime('%Y-%m')
portfolio_monthly.index = pd.to_datetime(portfolio_monthly.index).to_period("M")
portfolio_monthly

```

```

# Filter factor dates to match the asset
ff_factors_subset=ff_factors_monthly[ff_factors_monthly.index.isin(portfolio_monthly.index)].copy()

# Calculate returns for the asset

ff_factors_subset=pd.concat([ff_factors_subset,portfolio_monthly],axis=1)
ff_factors_subset.dropna(inplace=True)

ff_factors_subset

```

```

# Fama-French regression

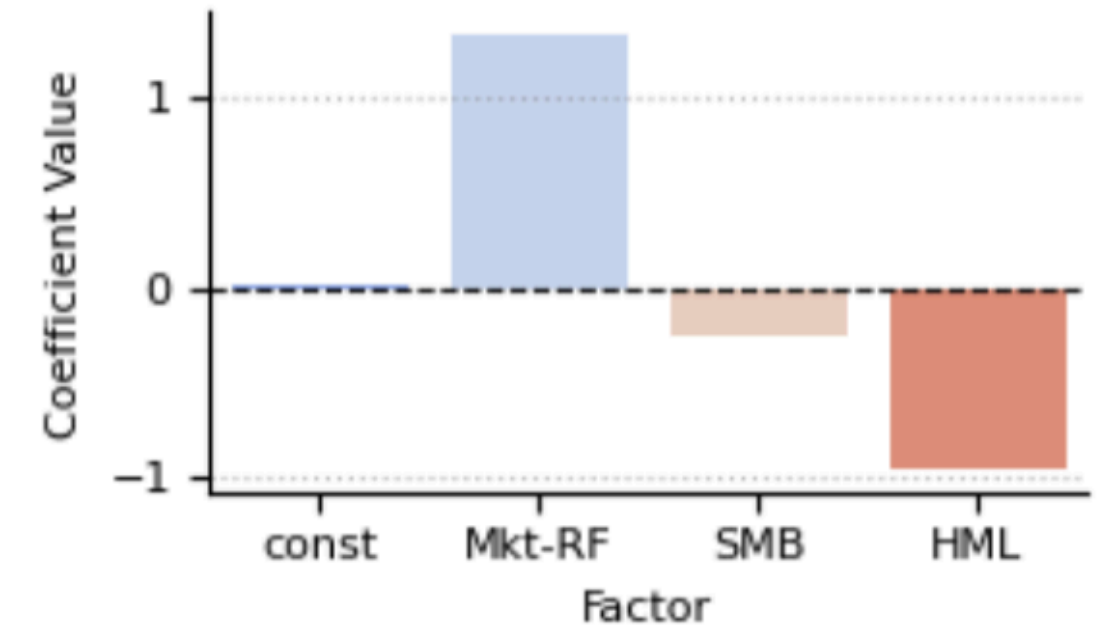
def fama_french_regression(returns,market,smb,hml,rf):
    X=sm.add_constant(pd.concat([market,smb,hml],axis=1))
    betas=[]
    for stock in returns.columns:
        Y=returns[stock]-rf
        model=sm.OLS(Y,X).fit()
        betas.append(model.params.values)
    betas_df=pd.DataFrame(betas,columns=['alpha','beta_mkt','beta_smb','beta_hml'],index=returns.columns)
    return betas_df,model

betas,model=fama_french_regression(returns,market,smb,hml,rf)

```

Code Implementation

Impact of Fama-French Factors on AMZN Returns



RESULTS

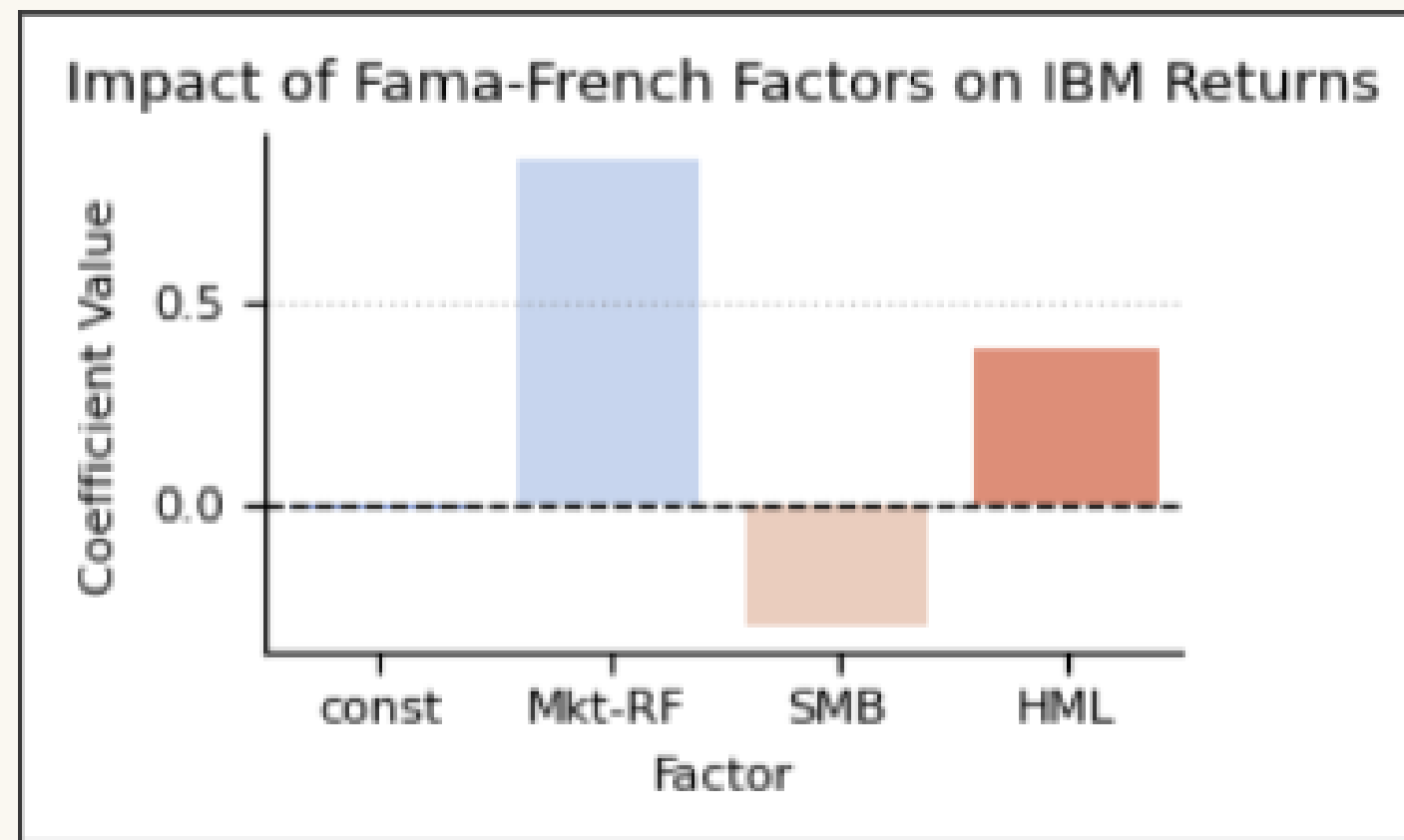
The Fama-French 3-factor regression estimates the impact of:

Market excess return (Mkt-RF)

Size (SMB: Small Minus Big)

Value (HML: High Minus Low)

The model quantifies how much each risk factor contributes to expected returns



Fama-French

- Fama-French: More realistic as it adds size (SMB) and value (HML) factors, capturing more variation in returns.

Hence, Fama-French Three-Factor Model consistently outperforms CAPM in explaining stock returns

DRAWBACKS:

- Still misses some anomalies (e.g., momentum)..
- Assumes factors are stable over time.
- Cannot capture sector specific anomalies and deviations with unique characteristics.
- Some correlation between the factors (e.g., small stocks might also tend to be value stocks) difficult separate their effects in certain market environments.

Markowitz's Mean-Variance Optimization (MVO)



Overview

MVO is a mathematical framework to construct an optimal portfolio by maximizing expected return for a given level of risk or equivalently, minimizing risk for a given expected return. The model uses the expected returns, variances, and covariances of assets to balance risk and return.

Formula Overview

- Minimize portfolio variance:

$$\sigma_p^2 = w^T \Sigma w \text{ where:}$$

- σ_p^2 is the portfolio variance,
 - w is the vector of portfolio weights,
 - Σ is the covariance matrix of asset returns.
- Subject to an expected return constraint: $E(r_p) = w^T E(r)$ where:
 - $E(r_p)$ is the expected portfolio return,
 - w is the vector of portfolio weights,
 - $E(r)$ is the vector of expected asset returns.



MVO - IMPLEMENTATION



```
# Expected Returns using historical mean of monthly returns
tickers = [
    "AAPL", "MSFT", "AMZN", "NVDA", "GOOGL", "TSLA", "JNJ", "WMT",
    "PEP", "CSCO", "LOW", "PM", "IBM", "UPS", "QCOM", "ORCL", "RTX", "NFLX"
]

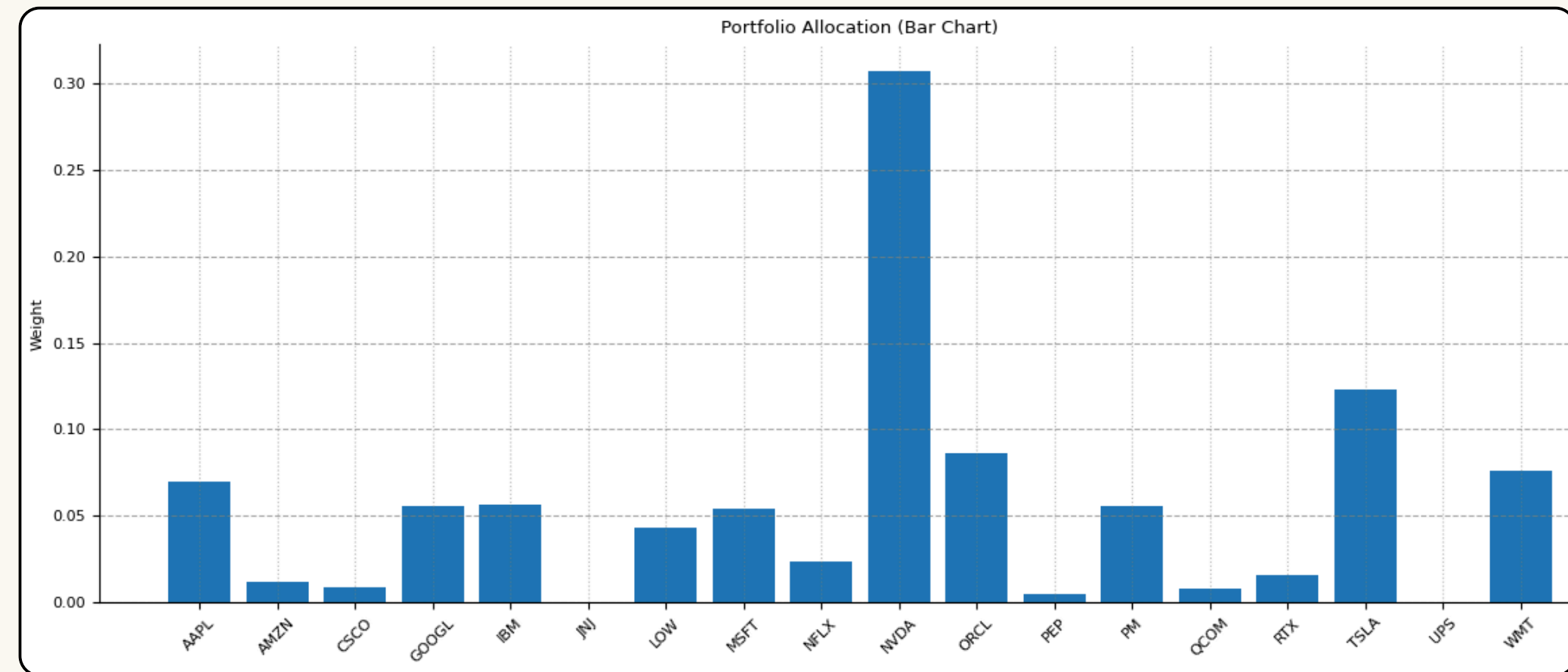
pf_m=yf.download(tickers,start="2020-01-01",end="2024-12-31",interval="1mo")["Close"]

mu = expected_returns.mean_historical_return(pf_m, frequency=12)

#covariance matrix
S = risk_models.sample_cov(pf_m, frequency=12)

# PyPortfolioOpt for portfolio optimization
ef = EfficientFrontier(mu, S)
ef.add_objective(objective_functions.L2_reg)
weights = ef.max_sharpe() # Optimize for max
cleaned_weights = ef.clean_weights() # Get c
```

Portfolio Allocation Bar Chart



```
expected_annual_return, annual_volatility, sharpe_ratio = ef.portfolio_performance
print(f"Portfolio Performance:")
print(f"Expected Annual Return: {expected_annual_return:.4f}")
print(f"Annual Volatility: {annual_volatility:.4f}")
print(f"Sharpe Ratio: {sharpe_ratio:.4f}")
```

```
Portfolio Performance:
Expected Annual Return: 0.4620
Annual Volatility: 0.2880
Sharpe Ratio: 1.6038
```

The **Expected returns** and **Covariance matrix** act as inputs to the efficient frontier and weights are allocated so as to **maximize sharpe ratio** (along with regularization so that there is diversification)

EFFECIENT FRONTIER

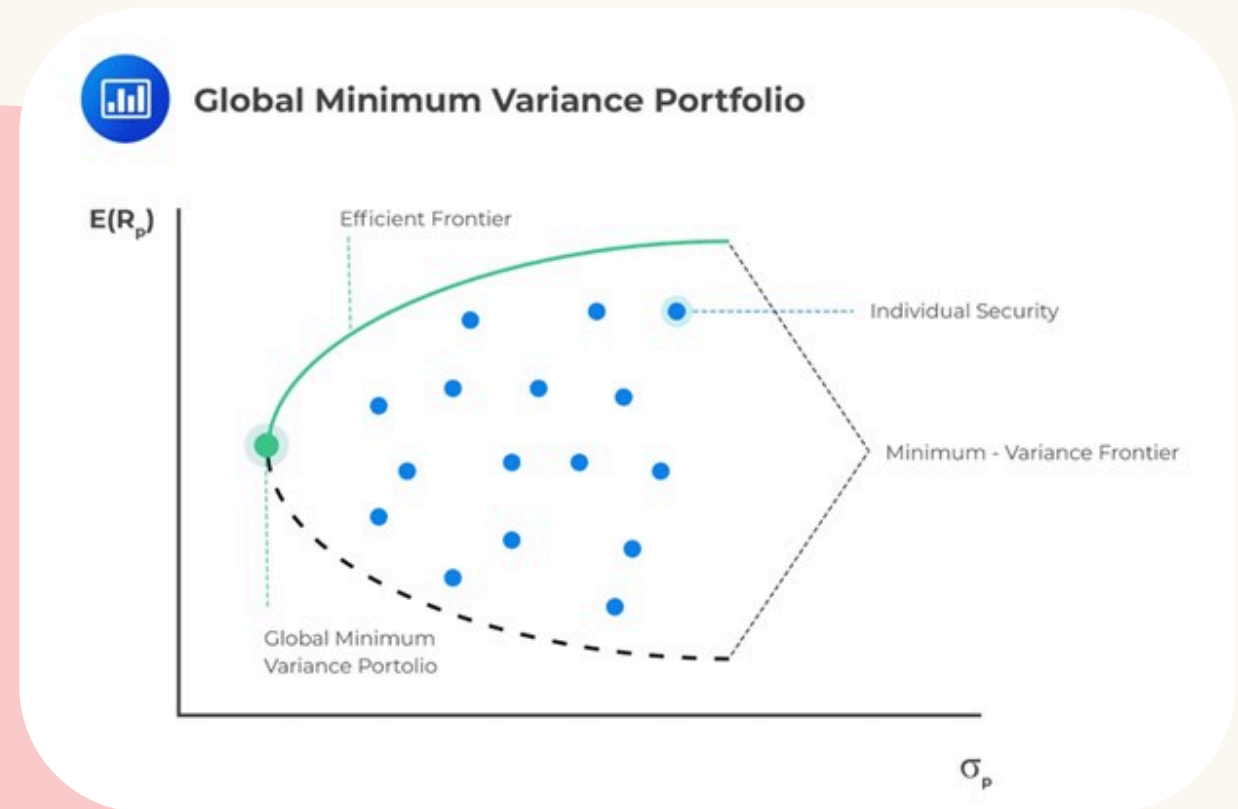
Overview

The efficient frontier represents the set of optimal portfolios that give:

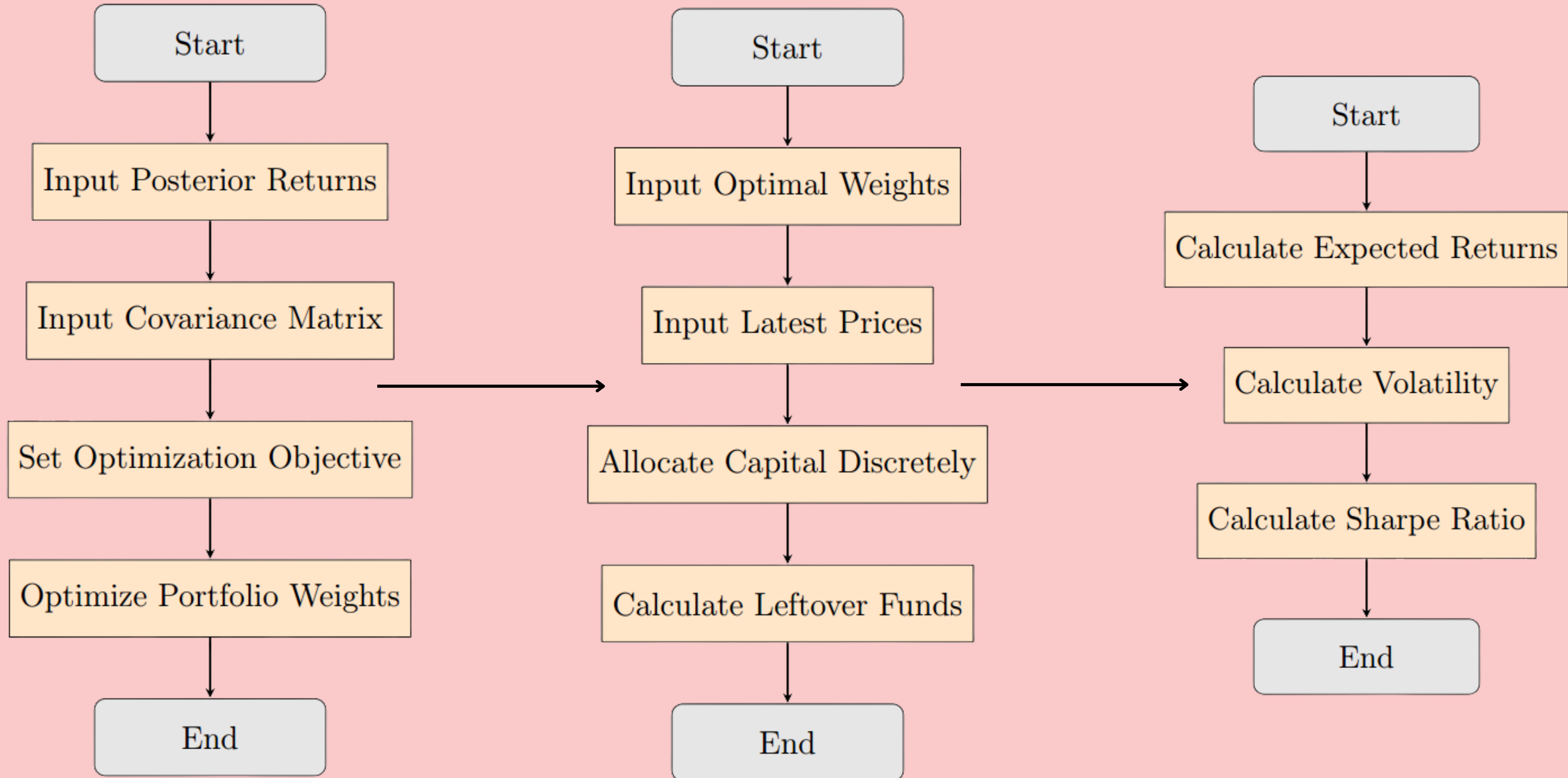
- Highest expected return for each risk level
- Lowest risk for each return level
- Best possible risk-return combinations
- Investment opportunity set boundary

Graph overview:

- Portfolios (points on frontier - green line) generally outperform individual securities and offer the best risk-return available
- One cannot increase expected returns($E(R_p)$) without accepting more risk (σ_p)
- Below the curve lie inefficient portfolios/stocks (blue dots) and above the curve we have impossible portfolio, i.e. more returns aren't possible with current set of stocks from which portfolio is made



Process Flowchart





Significance

- **Combines market equilibrium returns (like CAPM) with subjective investor views.**
- **Produces More Realistic Portfolios**
- **Flexibility**



Uses

- **Portfolio Optimization**– Used by institutional investors, quant funds, and portfolio managers to allocate assets more reliably
- **Scenario Analysis & Stress Testing**– Investors can model “what if” scenarios (e.g., recession, sector boom) by inputting custom views.
- **Risk Management**– Helps build portfolios that are more aligned with investor risk preferences and market consensus
- **Multi-Asset Allocation**– Useful across asset classes (stocks, bonds, commodities) in global macro portfolios.



Objective

The primary aim of this project is to design and implement a systematic investment strategy that integrates machine learning-based return prediction, fundamental analysis, and portfolio optimization.

Accurate Return Prediction: using ML model (XGBoost)

Quality Stock Selection: using fundamental factors

Risk-Optimized Portfolio Construction: using the Black-Litterman framework



Feature Pre-processing



For feature engineering, we have added and used the following indicators to train our ML model :

Price-Based Indicators

- Daily Returns – Percentage change in closing price.
- 20-Day Rolling Returns – Medium-term performance trend.

Trend Indicators

- Moving Averages – MA10 (short-term) and MA50 (medium-term) to capture momentum direction.
- Momentum – Percentage change in moving averages over 10 and 50 days.

Volatility Measures

- 20-Day Standard Deviation – Captures market uncertainty and price fluctuations.

Bollinger Bands

- Upper & Lower Bands – Identify overbought/oversold zones based on volatility.

Volume Relationship

- Correlation of Close Price & Volume (20-Day) – Detects demand-driven price changes.

Lagged Returns

- Past returns shifted by 1, 2, 3, and 5 days to capture short-term patterns and autocorrelation.



Feature Selection

Principal Component Analysis

It reduces dimensionality by transforming your original features into a new, smaller set of uncorrelated variables called principal components. During EDA, a scree plot helps determine the optimal number of components by identifying the "elbow point."

Random Forest

Mutual Information measures how much knowing one variable reduces uncertainty about another, capturing both linear and non-linear relationships. In feature selection, it identifies features that share the most information with the target variable. Higher scores indicate more informative features for prediction.

Mutual information

It combines multiple decision trees to make predictions, reducing overfitting through random sampling of data and features. It uses majority voting for classification and averaging for regression. Besides prediction, it is widely used to rank feature importance.



XGBoost Regressor

Purpose:

To predict the next-day returns of each stock based on engineered features.

Accurate Return Prediction: using ML model (XGBoost)

Why XGBoost?

- Handles non-linear relationships between indicators and returns.
- Built-in regularization reduces overfitting.
- Efficient for large datasets with many features.

Model Setup:

- Target Variable: Next-day return (%).
- Training: Separate model for each stock.
- Hyperparameters:
 - `n_estimators = 2000` (number of boosting rounds)
 - `max_depth = 12` (controls model complexity)
 - `learning_rate = 0.01` (slower learning, better generalization)
 - `subsample = 0.8` & `colsample_bytree = 0.8` (prevent overfitting)

```
model = xg.XGBRegressor(  
    objective='reg:squarederror',  
    n_estimators=2000,  
    max_depth=12,  
    learning_rate=0.01,  
    subsample=0.8,  
    colsample_bytree=0.8,  
    random_state=42  
)  
  
model.fit(  
    X, y,  
    eval_set=[(X, y)],  
    verbose=False,  
)
```

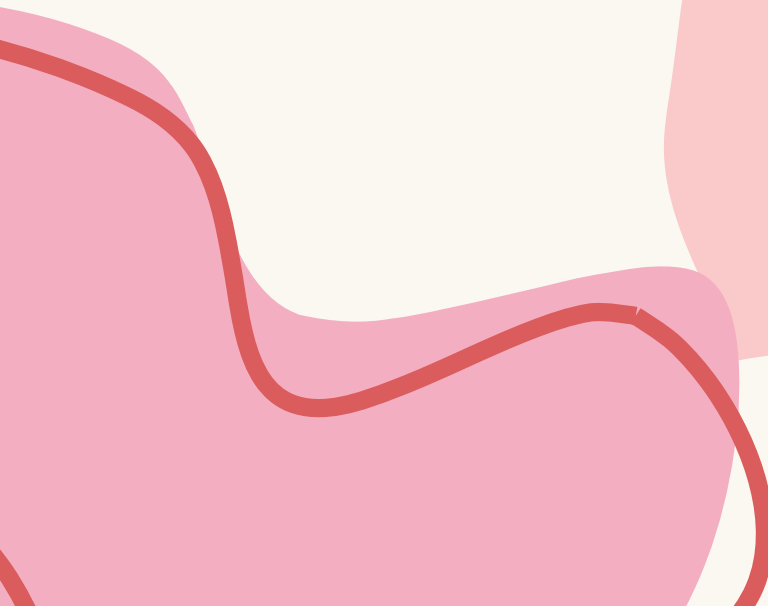


ARIMA MODEL

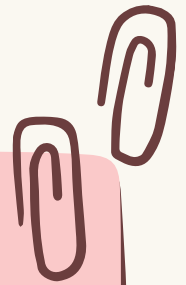


ARIMA stands for AutoRegressive Integrated Moving Average — it's a widely used statistical model for time series forecasting.

It predicts future values based on past observations and past prediction errors.



Black-Litterman Model



– Blending Market Views with Data-Driven Predictions

- Market equilibrium returns → Based on historical data .
- Our views → From XGBoost predictions of stock performance.
- Bayesian approach → Adjusts equilibrium returns toward our views depending on our confidence.
- The Black-Litterman formula is given below:
 - $E(R)=[(\tau\Sigma)^{-1}+PT\Omega^{-1}P]^{-1}[(\tau\Sigma)^{-1}\Pi+PT\Omega^{-1}Q]$
 -
- Optimization goal: Maximize Sharpe ratio.

Code Implementation



```
[61] def black_litterman_posterior(cov, pi, P, Q, Omega, tau=0.025):
    #  $\tau\Sigma$ : scaled prior covariance
    #  $\mu_{bl} = [(\tau\Sigma)^{-1} + P^T\Omega^{-1}P]^{-1} * [(\tau\Sigma)^{-1}\pi + P^T\Omega^{-1}Q]$ 
    #  $\Sigma_{bl} = [(\tau\Sigma)^{-1} + P^T\Omega^{-1}P]^{-1}$ 
    tau_cov = tau * cov
    inv_tau_cov = np.linalg.inv(tau_cov)
    inv_Omega = np.linalg.inv(Omega)

    M = np.linalg.inv(inv_tau_cov + P.T @ inv_Omega @ P)
    mu_bl = M @ (inv_tau_cov @ pi + P.T @ inv_Omega @ Q)
    return mu_bl, M

[38] mu_bl, cov_bl = black_litterman_posterior(cov_sample, pi, P, Q, Omega)
```

```
[62] #using PyPortfolioOpt
from pypfport.efficient_frontier import EfficientFrontier
def mean_variance_opt(mu, cov, allow_short=False):
    ef = EfficientFrontier(mu, cov)
    ef.max_sharpe()
    return ef.clean_weights()
```

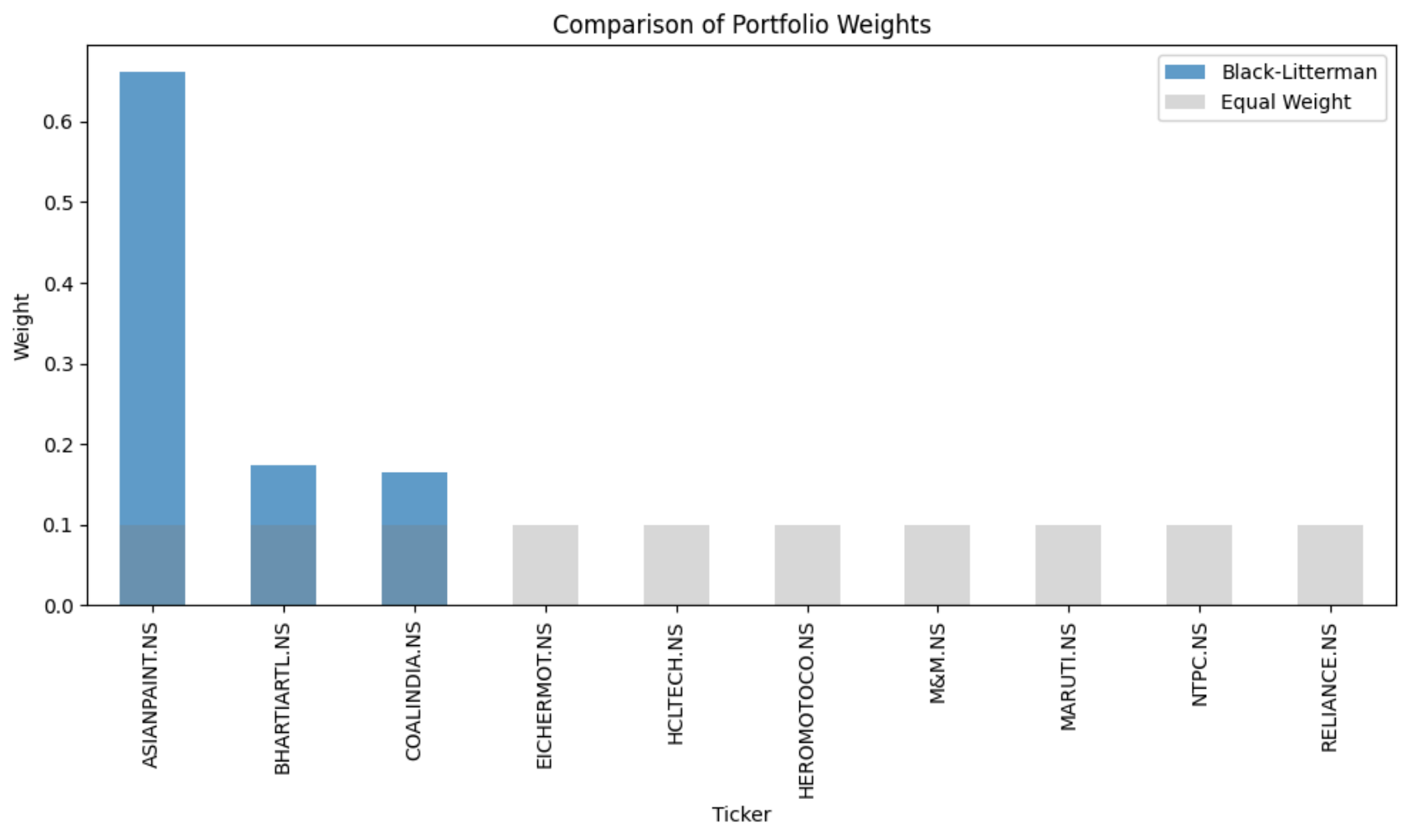
```
[63] optimal_weights = mean_variance_opt(mu_bl, cov_bl)
    optimal_weights = np.array(list(optimal_weights.items()))
    optimal_weights = np.array(list(optimal_weights[x][1] for x in range(len(optimal_weights))))
    optimal_weights
```

- Implied Equilibrium Returns:

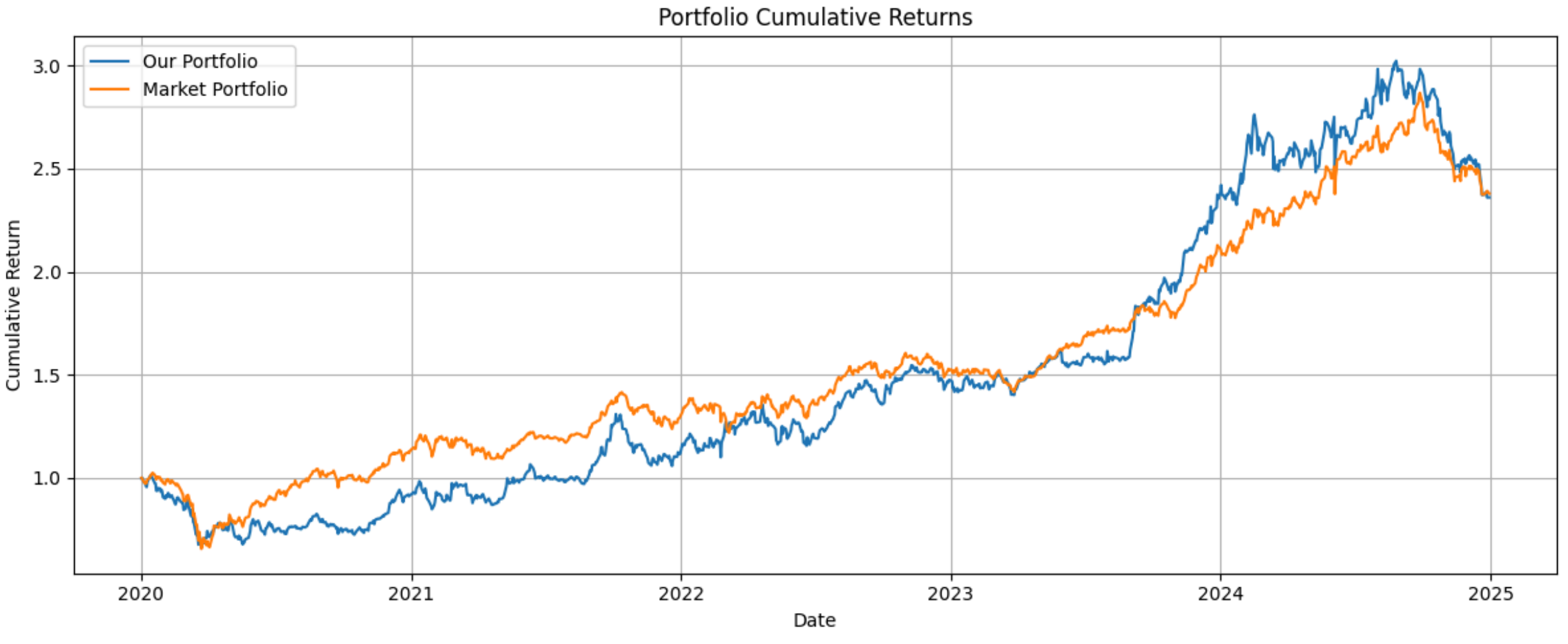
$$\Pi = \lambda \Sigma w_m$$

where:

- Π is the vector of implied equilibrium excess returns,
 - λ is the risk aversion coefficient,
 - Σ is the covariance matrix of returns,
 - w_m is the market capitalization weight vector of assets.
- Posterior Returns (after incorporating views): $\mu_{BL} = ((\tau\Sigma)^{-1} + P^T\Omega^{-1}P)^{-1} ((\tau\Sigma)^{-1}\Pi + P^T\Omega^{-1}Q)$ where:
 - μ_{BL} is the posterior expected returns,
 - τ is a scaling factor (usually a small positive number),
 - P is the matrix of views,
 - Ω is the covariance matrix of the views,
 - Q is the vector of view returns.



Results



Ticker	
BHARTIARTL.NS	0.18032
COALINDIA.NS	0.00000
HCLTECH.NS	0.51729
HEROMOTOCO.NS	0.00000
INFY.NS	0.30241
MARUTI.NS	0.00000
ONGC.NS	0.00000
RELIANCE.NS	0.00000
TCS.NS	0.00000
WIPRO.NS	0.00000

Work Experience

1ST PARAM

AR (AutoRegressive, p)
Uses the relationship between a current value and its own past values.
Example: today's stock price depends partly on yesterday's and the day before's prices

2ND PARAM

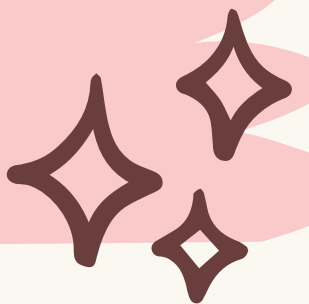
I (Integrated, d)
Refers to differencing the data to make it stationary (removing trends and seasonality).
If data has an upward trend, subtracting consecutive values (1st difference) can stabilize it.

3RD PART

MA (Moving Average, q)
Uses the relationship between the current value and past forecast errors.
Example: if the model overestimated yesterday, it adjusts today's prediction.

The background features soft, abstract pink shapes in the corners: a large one in the top right, a smaller one in the bottom left, and a faint one in the bottom right.

Two simple line-art paper clips are positioned to the left of the word 'Thank'.
Thank

You Three small, four-pointed star sparkles are located to the right of the word 'You'.